

PLAN DE REPARAȚII

RONOR — Sovereign Intelligence Operating Runtime

Registrul de lucrări, ordonat pe porți de dependență

Obiect	Plan de reparație și finalizare RONOR — pachete de lucru pe șapte porți de dependență
Data	25 august 2026
Revizia de referință	44f379870be43b617c4838cba0f066c053ce2b1a (origin/main)
Depozit	github.com/Constantin1968/RONOR-
Măsurători noi	Caddyfile citit integral (175 linii, v2.11.4), 17 căi sondate public
Gazde examinate	4 din 4 — DigitalOcean primar, Hetzner secundar, Contabo, bastion DO
Regim	Niciun merge, push, release sau deploy fără acord explicit
Clasificare	Intern — conține referințe la expuneri neremediate

Toate valorile din acest raport au fost măsurate direct, prin sondare live, interogare de protocol cu porturi de control, și citire directă a codului sursă. Nicio valoare de secret nu este afișată. Nicio operațiune de scriere nu a fost efectuată pe depozit sau pe sistemele de producție.

RONOR — Plan de reparație și finalizare

Registrul de lucrări, ordonat pe porți de dependență

Data: 25 august 2026 **Bază:** raportul de orientare din 25 august, plus măsurători noi făcute pentru acest plan — configurația Caddy citită integral de pe gazda Hetzner și sondarea publică a tuturor celor 17 căi pe care le publică. **Regim:** niciun merge, push, release sau deploy fără acordul dumneavoastră, cerut de fiecare dată. Nicio valoare de secret și niciun hash de parolă nu apar în acest document. **Ordonare:** exclusiv pe dependență. Nu apar estimări de timp, pentru că ordinea contează, nu durata.

0. Ce s-a schimbat față de raportul de orientare

Ca să construiesc planul pe configurația reală și nu pe deducție, am citit `/etc/caddy/Caddyfile` de pe Hetzner — 175 de linii, Caddy v2.11.4, `systemd`, `active` și `enabled`, `caddy validate` răspunde `Valid configuration`. Apoi am sondat public fiecare cale declarată.

Constatarea din raportul de orientare — „tabloul de bord e expus neautentificat” — era corectă, dar incompletă. Nu e o suprafață expusă. Sunt treisprezece, iar trei dintre ele sunt planuri de execuție și de inferență.

Sondaj public complet, <http://178.104.118.10>

Cale	Cod	Ce răspunde	Autentificare
<code>/gw/</code>	200	„AI Gateway says hey!” — Portkey AI Gateway	niciuna
<code>/temporal</code>	200	interfața web Temporal, 2.781 B	niciuna
<code>/langgraph/</code>	200	<code>{"ok":true}</code>	niciuna
<code>/guardrails/</code>	200	banner de serviciu cu lista de endpointuri	niciuna
<code>/nemo/</code>	200	banner de serviciu	niciuna
<code>/lakera/</code>	200	banner de serviciu	niciuna
<code>/cida/</code>	200	descriptor complet, cele șase niveluri	niciuna la margine
<code>/comms/health</code>	200	<code>smtp_configured:true, imap_configured:...</code>	niciuna
<code>/memory/health</code>	200	<code>qdrant_ok:true, collection:ronor...</code>	niciuna
<code>/dashboard/</code>	200	RONOR Command Center	niciuna
<code>/qdrant/collections</code>	401	„Must provide an API key or an Authorization bearer token”	la nivel de aplicație — corect
<code>/minio/</code>	401	—	<code>basic_auth</code> în Caddy — corect
<code>/control/</code>	401	—	<code>basic_auth</code> în Caddy — corect
<code>/crewai/, /r-monitor/, /r-execute/, /r-schedule/</code>	404	<code>{"detail":"Not Found"}</code>	rută montată, serviciu fără rădăcină

Constatări noi de expunere

E-01 — Poarta de modele este publică. `POST /gw/v1/chat/completions` fără nicio credențială returnează **400** cu mesajul `Either x-portkey-config or x-portkey-provider header is required`. Aceasta este dovada decisivă: cererea a trecut de stratul de autentificare și a eșuat abia la validarea antetelor de rutare. Nu există autentificare la marginea porții. Cine cunoaște sau ghicește un identificator de configurație Portkey poate emite cereri către furnizorii dumneavoastră, pe bugetul dumneavoastră. Aceasta este cea mai costisitoare expunere din sistem, pentru că se traduce direct în bani.

E-02 — Planul de orchestrare este public. `GET /temporal/api/v1/namespaces` returnează 200 și dezvăluie namespace-ul `ronor`, starea `NAMESPACE_STATE_REGISTERED` și identificatorul `04069c66-b35e-4836-9baf-cd1691983c2e`. Interfața Temporal servește pe aceeași cale. Un plan de orchestrare a fluxurilor de lucru nu se publică pe internet.

E-03 — Planul de raționament acceptă scrieri neautentificate. `POST /langgraph/assistants/search` cu corp JSON returnează 200 și dezvăluie asistentul `ronor_reasoner`, `graph_id` și data creării, 14 august. API-ul LangGraph acceptă cereri de tip scriere fără credențială. Nu am testat crearea unei execuții, deliberat — ar fi însemnat să pornesc muncă reală și să consum credit. Faptul că o cerere de căutare de tip `POST` e acceptată e suficient ca dovadă a suprafeței.

E-04 — Stratul de siguranță este el însuși deschis. `/guardrails/`, `/nemo/` și `/lakera/` își publică bannerele și listele de endpointuri. Componentele care ar trebui să apere sistemul de injecție de prompt și de jailbreak sunt ele însele accesibile public, deci pot fi sondate, calibrate împotriva dumneavoastră sau saturate.

E-05 — Componenta de comunicare confirmă public configurația. `/comms/health` declară `smtp_configured:true` și `imap_configured`. Acesta este exact containerul `ronor-r-comms` care deține tokenul botului Telegram și expune `POST /telegram/send` neguvernate. `/memory/health` confirmă `qdrant_ok:true` și numele colecției.

E-06 — Tabloul de bord nu a fost niciodată protejat. În Caddyfile, `handle /dashboard*` nu are `basic_auth`, în timp ce `/minio/`, `/cida-lake/` și `/control*` îl au. Nu e o regresie, e o omisiune de la început.

Constatări favorabile, noi și importante

F-A — control.ronor.tech există și e făcut corect. Un bloc de site propriu, cu TLS automat prin ACME, `basic_auth`, și un set complet de anteturi de securitate: `Strict-Transport-Security` pe un an cu subdomenii, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, și suprimarea antetului `Server`. Comentariul din configurație explică și de ce proxy-ul Cloudflare e dezactivat deliberat pe înregistrarea A — ar intercepta validarea ACME.

Aceasta schimbă Poarta doi. **Domeniul `ronor.tech` este al dumneavoastră și e deja operațional.** Planul de interfețe are o casă reală. Nu trebuie inventată.

F-B — admin off. Endpointul de administrare Caddy este dezactivat. Corect.

F-C — Există un obicei de backup. Unsprezece copii `Caddyfile.bak.*` date între 5 și 9 august. Cine a lucrat pe acest fișier a făcut-o cu grijă.

F-D — Modelul corect de protecție există deja în casă. Trei căi sunt protejate cu `basic_auth`, iar Qdrant refuză la nivel de aplicație. Nu trebuie proiectat un mecanism nou. Trebuie extins cel existent.

1. Principiile de execuție ale acestui plan

Cinci reguli pe care mi le impun, ca planul să nu producă un blocaj mai mare decât problema pe care o repară.

- Nicio schimbare fără cale de întoarcere demonstrată.** Fiecare pachet de lucru are un pas de backup înainte și o comandă de revenire explicită, testată logic pe configurația citită.
- Nicio schimbare fără verificare empirică după.** Nu declar un pachet încheiat pe baza faptului că fișierul s-a scris. Îl declar încheiat când sondarea publică dă răspunsul așteptat. Motivul e concret: ordinea de evaluare a directivelor în Caddy nu se deduce din citirea configurației, se dovedește prin apel.
- Nimic nu se atinge pe planurile Python Gen 1.5 care rulează.** Ele țin lumina aprinsă. Se protejează la margine, nu se modifică înăuntru.
- Merge, push, release și deploy rămân umane.** Cer acordul de fiecare dată, separat, cu descrierea exactă a ce se schimbă.
- Dacă un pas eșuează, mă opresc și raportez ce a eșuat și de ce.** Nu improvizez în jurul unei premise nedovedite.

Regimul de aprobare, pe trei niveluri

Nivel	Ce cuprinde	Regim
A — citire	sondări HTTP, citire de configurații și cod, inventar, <code>git log</code>	execut fără să întreb

Nivel	Ce cuprinde	Regim
B — schimbare reversibilă pe gazdă	Caddyfile cu backup și reload, declarare de container în compose, înregistrare de comenzi Telegram	cer acordul o dată per pachet, apoi execut integral și raportează
C — ireversibil sau cu efect asupra istoriei	rotație de chei, merge, push pe <code>main</code> , release, deploy Gen 2, ștergere	cer acordul explicit, de fiecare dată, cu descrierea exactă

2. Poarta zero — oprirea expunerii

Aceasta precede orice altă muncă. Nu pentru elegantă, ci pentru că `/gw/` deschis costă bani în fiecare oră în care rămâne deschis, iar restul porților construiesc peste o fundație care sângerează.

P0-1 — Închiderea celor treisprezece căi la margine

Nivel B. Aceasta este singura intervenție care rezolvă E-01 până la E-06 dintr-o dată, fără să atingă niciun serviciu.

Principiul: căile operaționale nu au ce căuta pe internetul public. Ele sunt instrumente de arhitect, iar arhitectul intră prin tailnet. Se restrâng la spațiul Tailscale `100.64.0.0/10`, la rețeaua Docker `172.20.0.0/16` și la localhost. Ce rămâne public: `cidavault.com`, `control.ronor.tech`, și decizia separată despre rădăcină de la P0-2.

Construcția, de introdus în blocul `:80`, imediat după `encode zstd gzip`:

```
# Plan operațional — accesibil doar din tailnet și din rețeaua internă.
# Introdus pentru închiderea expunerilor E-01...E-06.
route {
  @operational_public {
    path /gw/* /temporal* /langgraph/* /crewai* \
        /r-monitor/* /r-execute/* /r-schedule/* \
        /guardrails/* /nemo/* /laker/* \
        /comms* /memory* /dashboard* /qdrant/*
    not remote_ip 100.64.0.0/10 172.20.0.0/16 127.0.0.1/8
  }
  respond @operational_public "Forbidden" 403 {
    close
  }
}
```

Un `route` garantează evaluare secvențială, iar un matcher numit cu două condiții le combină prin conjuncție: cale în listă și sursă din afara spațiului intern. Ce nu se potrivește cade prin, către directivele existente, neatins.

Procedura, în ordine:

1. `cp /etc/caddy/Caddyfile /etc/caddy/Caddyfile.bak.pre-P0-1-$(date +%Y%m%d_%H%M%S)`
2. inserarea blocului, prin editare punctuală, fără rescrierea fișierului — credențialele `basic_auth` existente nu se ating și nu se citesc
3. `caddy validate --config /etc/caddy/Caddyfile` — trebuie să răspundă `Valid configuration`
4. `systemctl reload caddy` — reload, nu restart; conexiunile în curs nu se rup
5. verificare, obligatorie

Verificarea, din afară:

```

/gw/           → 403      (era 200)
/temporal      → 403      (era 200)
/langgraph/    → 403      (era 200)
/dashboard/    → 403      (era 200)
/comms/health  → 403      (era 200)
/memory/health → 403      (era 200)
/guardrails/   → 403      (era 200)
/cida/         → 200      neschimbat, are cheie proprie
/control/      → 401      neschimbat
cidavault.com  → 200      neschimbat

```

Verificarea, din interior, prin `ssh` pe gazdă: `curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1/dashboard/` trebuie să dea `200`. Dacă dă `403`, matcherul de sursă e prea strict și se corectează.

Revenirea: `cp` din backup, `caddy validate`, `systemctl reload caddy`. Un pas, câteva secunde.

Riscul asumat: dacă vreun serviciu intern apelează una dintre aceste căi prin adresa publică în loc de `127.0.0.1`, se va rupe. Se detectează imediat, în jurnalele Caddy, la pasul de verificare, și se rezolvă adăugând sursa în matcher.

Nota de onestitate: poziția `route` în ordinea implicită a directivelor Caddy este ceea ce fac aici să fie corect. Nu o afirm din memorie. De aceea pasul cinci nu e opțional, iar dacă sondarea nu dă `403`, construcția se schimbă în varianta cu `handle` explicit înaintea celorlalte, nu se declară reușită.

P0-2 — Rădăcina publică

Nivel B, dar cere o decizie a dumneavoastră. Astăzi `handle { reverse_proxy 127.0.0.1:3000 }` trimite orice cale nepotrivită la Langfuse, deci rădăcina publică a RONOR e pagina de autentificare a unui instrument de observabilitate. Trei variante, toate reversibile:

Variantă	Ce vede publicul la rădăcină	Efect
a. Langfuse intră în plan operațional	<code>403</code>	cel mai simplu, cel mai auster; se adaugă doar <code>/</code> la lista din P0-1
b. Langfuse se mută pe cale proprie protejată	<code>403</code> la rădăcină, Langfuse la <code>/langfuse/*</code> restrâns la tailnet	curat, dar Langfuse rescrie căi și poate cere ajustare
c. Rădăcina servește site-ul public din repository	site-ul RONOR, 880 de linii, deja construit	rezolvă simultan I-01 și prima jumătate din Poarta doi

Recomandarea mea: **a** acum, **c** la Poarta doi. Motivul: Poarta zero e despre oprirea sângerării, nu despre a arăta bine. Varianta **c** e o schimbare de suprafață publică și merită luată ca decizie de interfață, nu ca reparație de urgență.

P0-3 — Rotația cheilor de API RONOR

Nivel C. Codul vă cere el însuși acest lucru: `src/runtime/api/routes.ts`, linia 234, emite `insecure-default-key: a shipped default API key is active – rotate RONOR_API_KEYS`. Peste asta, două chei statice fără expirare, cu etichetele `telegram-bridge:` și `console:`, au trecut prin contextul meu din cauza eșecului șablonului meu de redactare. Nu le-am reproduș și nu le voi reproduce, dar au fost expuse și trebuie tratate ca atare.

Ordinea corectă, ca să nu se rupă nimic: generare de chei noi pe gazdă → adăugare lângă cele vechi, ambele valide → actualizarea consumatorilor, `telegram-bridge` și consolă → verificare că totul răspunde → eliminarea cheilor vechi. Nu invers. Rotația simultană rupe consumatorii.

Cere acordul dumneavoastră explicit, pentru că e ireversibilă și atinge autentificarea.

P0-4 — Codurile de recuperare GitHub de pe laptop

Nivel B, execut la cerere. `github-recovery-codes.txt`, 206 B, se află în clar în `C:\Users\Hp\Downloads`, lizibil de grupul `CodexSandboxUsers` cu `ReadAndExecute`. Alături, `env.automation.corrected` și un `.env` cu parole. Acțiunea minimă: mutare într-un director fără acces de grup și restrângerea listei de control al accesului. Acțiunea corectă: regenerarea codurilor la GitHub și păstrarea lor în afara discului — dar consola GitHub nu se accesează de pe IP rusesc, deci pasul acesta rămâne al dumneavoastră, de pe o ieșire potrivită.

P0-5 — Cele trei expuneri deschise din raportul anterior

Nivel B și C. Rămân neatinsse și nu le-am reverificat pentru acest plan: C-01 Whisper public neautentificat pe `169.58.129.223:8200`, C-02 Postgres 16 public pe `165.245.248.223:5432`, C-03 ocolirea `ufw` de către Docker. Primele două sunt pe gazde diferite de Hetzner. C-02 e pe DigitalOcean, care nu răspunde HTTP public dar are Postgres deschis — combinația cea mai proastă posibilă: nefolositoare pentru dumneavoastră, folositoare pentru altcineva.

Poarta zero se declară închisă când sondarea publică a tuturor celor patru gazde nu mai returnează niciun 200 neautentificat pe o cale operațională, și când cheile din P0-3 sunt rotite.

3. Poarta unu — recâștigarea cunoașterii asupra a ceea ce rulează

Nu se poate governa ce nu e versionat, și nu se poate finaliza ce nu e cunoscut. Trei artefacte rulează în producție și nu există în niciun repozițoriu.

P1-1 — `/opt/ronor-cc` sub control de versiune

Nivel B. Tabloul de bord React — Vite, React, Tailwind, plus `server/` — datat 5 august, fără `.git`, `node server/index.js` pid 932, unsprezece zile de funcționare, fără supervisor. E singura interfață care răspunde efectiv și e cea mai fragilă componentă din sistem: o repornire de gazdă îl oprește definitiv, iar sursa există într-un singur loc, pe un disc.

Acțiunea nu e refactorizare. E înregistrare: `git init`, un commit inițial cu starea exactă, apoi decizia de la Poarta doi despre unde trăiește. Separat, un serviciu `systemd` cu `Restart=always`, ca să nu depindă de un proces orfan.

P1-2 — Planurile Python Gen 1.5 sub control de versiune

Nivel B. `/opt/ronor/*.py`, `/opt/ronor-planes`, agregatorul de telemetrie de pe 8401, serviciul de pool de pe 8402, `r-comms`, `r-monitor`, `r-execute`. Rulează de peste unsprezece zile și nu sunt nicăieri. Același principiu: se înregistrează, nu se rescriu. Nu se atinge nimic din comportament.

P1-3 — Configurațiile desfășurate ale containerelor

Nivel B. Containerele Telegram care rulează — și cel care a murit — nu sunt declarate în niciun fișier compose de pe gazde, deși `docker-compose.production.yml` din repozițoriu le declară la liniile 219–252. Discrepanța asta explică de ce cincisprezece zile de moarte a botului au trecut neobservate. Se extrage configurația efectivă din `docker inspect` și se scrie într-un compose real.

P1-4 — Contabo: acces sau declarare de pierdere

Nivel B pentru diagnostic, decizia e a dumneavoastră. A treia gazdă refuză cheia — `Permission denied (publickey,password)` — și propriul tablou de bord o raportează `provisioning`. Nu o controlați. Două ieșiri oneste: recuperarea accesului prin consola furnizorului, care nu se accesează de pe IP rusesc și deci e a dumneavoastră, sau declararea ei drept pierdută și scoaterea din inventar. A treia variantă, cea de acum — o gazdă necunoscută trecută în inventar ca activă — este singura inacceptabilă, pentru că e și suspectul principal pentru conflictele `getUpdates` de la Poarta trei.

P1-5 — Reconcilierea celor trei stări ale codului

Nivel A, execut oricând. `main` pe GitHub la 44f3798, laptopul pe `fix/evidence-runner-child-env` la bcd9f9f, DigitalOcean pe `fix/release-readiness-conf5` la 6a50a7e, cu aproximativ 25 de commituri în urmă. Nicio gazdă nu rulează `main`. Produc un tabel de divergență: ce e în cele șaisprezece ramuri locale, ce e terminat, ce e abandonat, ce trebuie integrat înainte de Poarta cinci. Livrabil de citit, nu de executat.

Poarta unu se declară închisă când fiecare proces care rulează în producție are o sursă versionată corespunzătoare, un supervisor, și un răspuns clar la întrebarea „ce revizie e asta”.

4. Poarta doi — decizia de interfețe, scrisă în repozițoriu

Aici planul se oprește și așteaptă o decizie care e a dumneavoastră, nu a mea. Nu pot decide în locul dumneavoastră care e suprafața principală, pentru că e o decizie de produs, nu de inginerie. Dar pot spune care sunt opțiunile și ce costă fiecare.

Doctrina din 6 august e sănătoasă și, acum că știu că **ronor.tech** e al dumneavoastră și **control.ronor.tech** funcționează cu TLS propriu, e și implementabilă fără muncă nouă de infrastructură: **o suprafață principală de comandă, Telegram ca canal mobil de aprobare, o singură bază de cunoaștere.**

P2-1 — Decizia care blochează tot restul

Nivel: decizie. Două suprafețe candidează la titlul de suprafață principală și nu pot coexista amândouă:

	Tabloul de bord React <code>/opt/ronor-cc</code>	Consola de operator din repository
Stare	rulează, unsprezece zile	construită, 2.529 de linii, nedesfășurată
Versionare	inexistentă	în <code>main</code>
Backend	propriu, <code>server/index.js</code>	montat de <code>src/index.ts</code> linia 262
Cost dacă e aleasă	aducere în repository, supervisor	desfășurare, plus verificarea că apelurile ei corespund rutelor reale
Cost dacă e respinsă	se retrage un lucru care funcționează	se abandonează 2.529 de linii scrise

Două suprafețe principale înseamnă zero. Recomandarea mea, dacă o vreți: **consola din repository devine suprafața principală, tabloul de bord React devine ecranul de stare al patrimoniului sub ea.** Motivul e că guvernanta cere versionare, iar consola e deja în `main`, în timp ce tabloul de bord e un artefact orfan. Dar dacă tabloul de bord face în fapt lucruri pe care consola nu le face, decizia se inversează, și doar dumneavoastră știți asta.

P2-2 — `/control` să servească interfața de arhitect din repository

Nivel B. Astăzi `/control*` e protejat corect cu `basic_auth` și trimite la agregatorul Python de telemetrie de pe 8401. Interfața de arhitect din repository — `web/control/`, 101 linii — nu e servită nicăieri. Sunt două lucruri diferite cu același nume, iar asta e o capcană pe termen lung.

P2-3 — Cheia de arhitect iese din `sessionStorage`

Nivel C. `docs/control-executive-council.md` spune că cheia „nu trebuie stocată niciodată în Git, într-un URL, în stocarea locală a browserului, într-un email sau în starea unei misiuni”. `web/control/control.js` o stochează în `sessionStorage['ronor.control.key']`. Documentația are dreptate, codul o contrazice. Se aliniază codul la doctrină, nu invers.

Separat, aceeași secțiune are o divergență de rute: documentația descrie `/api/runtime/management/*`, clientul apelează `/api/runtime/control/*`. Ambele există în `routes.ts`, la liniile 609 și 833. Una dintre ele e moartă și trebuie declarată ca atare.

P2-4 — „RONOR App” se declară sau se elimină din vocabular

Nivel: decizie. Astăzi e un cuvânt fără referent: zero manifest, zero service worker, zero meta pentru mobil. Cerința de teren formulată pe 26 iulie — „interfață umană, pe telefon, când ești pe teren: vezi rezultate, aprobi decizii, delegi misiuni noi” — a rămas neimplementată. Trei ieșiri:

- **Mini App Telegram.** Cel mai puțin efort, pentru că vine cu autentificarea și cu canalul deja rezolvate, și pentru că botul e oricum canalul mobil din doctrină. Cere HTTPS, pe care **control.ronor.tech** îl are deja.
- **Aplicație web progresivă** pornind din suprafața principală. Mai multă muncă, dar independentă de Telegram.
- **Eliminare din vocabular.** Perfect onorabilă. Canalul mobil e Telegram, iar „App” nu descrie nimic. Un termen care nu are referent produce confuzie în fiecare conversație viitoare.

P2-5 — Planul se scrie în repository

Nivel C, cere push. Aceasta e lecția centrală din întreaga radiografie. Doctrina exista din 6 august și era bună. Nu a fost scrisă niciodată în repository, și de aceea codul a evoluat fără ea, iar acum aveți patru suprafețe fără plan. Un document —

`docs/interface-plan.md` — care declară: care e suprafața principală, ce servește fiecare cale, care e canalul mobil, ce înseamnă „App”, și care suprafață e retrasă. Fără el, Poarta doi se redeschide la fiecare sesiune.

Poarta doi se declară închisă când există un document în `main` care răspunde la aceste patru întrebări, și când configurația Caddy îl reflectă.

5. Poarta trei — însănătoșirea și activarea botului

Botul `8885653110, @ronor_sovereign_bot`, e mort pe ambele gazde de pe 10 august. Ordinea de mai jos e strictă. O repornire înainte de pașii unu și doi produce un al doilea conflict `getUpdates` și pierdeți din nou vizibilitatea.

P3-1 — Localizarea sau excluderea celei de-a treia instanțe

Nivel A. Conflictele `409` s-au produs *după* ce containerul de pe DigitalOcean a fost oprit deliberat, pe 8 august la 23:53:07Z. Deci ceva a interogată Telegram pe 9 și 10 august, iar acel ceva nu e niciunul dintre cele două containere cunoscute. Contabo e suspectul care nu a putut fi verificat, ceea ce leagă acest pachet de P1-4. Fără răspuns aici, orice pornire e un pariu.

P3-2 — De-duplicarea tokenului

Nivel C. `ronor-r-comms` — imaginea `ronor/r-comms:1.0.0`, unsprezece zile de funcționare, `127.0.0.1:8100` — deține același token și expune un `POST /telegram/send` fără nicio guvernanta. Consecința e că RONOR poate vorbi pe Telegram, dar nu poate asculta. Canalul de aprobare e unidirecțional, adică nu e un canal de aprobare. Două ieșiri: token separat pentru `r-comms`, sau acceptarea explicită a credențialei partajate ca decizie de risc consemnată în scris. A treia variantă, cea de acum — partajare tacită — nu e o decizie, e o scăpare.

P3-3 — Contradicția de configurare

Nivel B. `TELEGRAM_MODE=polling` apare de două ori, iar variabilele de webhook sunt și ele setate. Server-side am verificat: niciun webhook înregistrat, zero actualizări în așteptare. Configurația trebuie să spună un singur lucru.

P3-4 — Declararea containerului și o politică reală de repornire

Nivel B. Containerul de pe Hetzner avea `RestartPolicy: no`. A primit `SIGTERM` pe 10 august la 01:19:33Z, a ieșit cu cod 0, și nimeni nu a observat cincisprezece zile. `restart: no` e explicația completă. Se declară în `compose`, cu `restart: unless-stopped` și o verificare de sănătate.

P3-5 — Înregistrarea comenzilor

Nivel B. Codul definește opt comenzi. La Telegram sunt înregistrate zero. Chiar când botul va rula, va fi invizibil în interfață — nu apare meniul, nu apare autocompletarea. `setMyCommands` e un apel, nu un proiect.

P3-6 — Separarea autorității de co-semnare

Nivel C. Mecanismul Poarta 1 / Poarta 2 din `approval-store.ts` este bine construit: TTL de 60 de minute, fără token de ocolire, re-trimiterea completă a payload-ului. Dar toate trei variabilele — `TELEGRAM_ALLOWED_USER_IDS`, `TELEGRAM_APPROVER_USER_IDS`, `TELEGRAM_CONTROL_CHAT_ID` — conțin același identificator, `7200344419`. Cu un singur semnatar, co-semnarea nu semnifică nimic. E teatru de guvernanta.

Aici Telegram v-a rezolvat problema: **mesajele efemere din Bot API 10.2** fac promptul de co-semnare vizibil doar co-semnatarului, într-un grup, fără să expună payload-ul întregului grup. Separarea devine practicabilă fără infrastructură nouă. Cine e al doilea semnatar e o decizie a dumneavoastră, nu o problemă tehnică.

P3-7 — Unificarea ponderilor de guvernanta

Nivel B. Hetzner: `QUALITY .25`, `COST .25`, `SOVEREIGNTY .25`, `LATENCY .10`, `RISK .10`, `EVIDENCE .05`, sumă 1.00. DigitalOcean: `QUALITY .35`, `COST .25`, `LATENCY .20`, `RISK .10`, sumă 0.90, **fără SOVEREIGNTY și fără EVIDENCE**. Cele două gazde nu decid după aceeași constituție, iar cea care ignoră suveranitatea și dovada e cea care nu adună nici măcar la unu. Într-un sistem al cărui nume începe cu „sovereign”, asta nu e o scăpare de configurare, e o contradicție de fond.

Favorabil: plafoanele PB-SEC-001 sunt configurate corect — cost maxim de misiune 2,00 USD, maximum 6 sarcini, timeout de agent 180 s, limită de preluare 2 MB, 3 încercări de rezervă, politica MI9 montată în container.

Poarta trei se declară închisă când botul răspunde la cele opt comenzi, cere co-semnare de la doi identificatori distincți, are un supervisor care îl repornește, și e declarat în compose cu revizie cunoscută.

6. Poarta patru — paritatea perceptuală cu reperul Grok

Numai după ce botul rulează guvernare. Ordonate după raportul dintre efect și efort, de la cel mai profitabil.

#	Intervenție	Efect	Efort
1	<code>sendMessageDraft</code> cu <code>draft_id</code> stabil, plus <code>sendMessage</code> final la încheiere	cea mai mare diferență percepută din toată lista	mic
2	Text gol în draft — placeholder „Thinking...” nativ	vizibil imediat	aproape nul
3	<code>can_stop=True</code> plus handler pe <code>stopped_message_generation</code>	control real al utilizatorului	mic
4	Butoane <code>style: danger</code> și <code>success</code> , <code>DisabledButton</code> la expirarea TTL	claritate pe fluxul de aprobare existent	mic
5	Memorie de conversație per chat, separată în reguli durabile față de instrucțiuni de sarcină	continuitate	mediu
6	Bucă de unelte către runtime-ul propriu	capabilitate reală, nu percepție	mediu
7	Rich Messages pentru output de guvernare — tabele, checkbox-uri, <code>details</code>	rapoartele arată ca rapoarte	mediu
8	Voce în ambele sensuri, prin STT și TTS externe	acoperă cerința de teren	mare

Constrângerile reale de proiectare, nu opționale: un mesaj pe secundă per chat, douăzeci pe minut în grup, `callback_data` limitat la 64 de octeți — deci indexuri scurte către stare, niciodată payload-uri. Descărcarea de fișiere e plafonată la 20 MB fără un Local Bot API Server propriu. Transcrierea vocală nu există în Bot API, deci lanțul e `getFile` → STT extern → model → TTS → `sendVoice`.

Ce nu se poate replica și nu merită urmărit: mașina virtuală cloud persistentă cu browser și terminal — esența Grok Bot, fără legătură cu Bot API. Preluarea controlului desktopului pentru parole, 2FA și CAPTCHA. Conversația vocală full-duplex. Streamingul în grupuri, pentru că draft-urile cer chat privat.

Unde depășiți reperul, fără muncă nouă: mandat semnat server-side, evidence runner izolat, atestare obligatorie, co-semnare cu TTL fără token de ocolire. Grok Bot nu are audit trail per-acțiune, nu are sandbox — un „test run” execută muncă reală — și setul de documente nu conține nicio revendicare SOC 2, ISO 27001, GDPR sau HIPAA. Pe guvernare și dovadă, reperul e sub dumneavoastră.

Poarta patru se declară închisă când un utilizator care nu știe ce rulează dedesubt nu poate distinge botul de un asistent comercial de primă linie.

7. Poarta cinci — desfășurarea Gen 2

1.084 de teste trecute, zero erori de compilare, cel mai riguros strat de autoritate din tot sistemul — și nedesfășurat pe nicio gazdă.

P5-1 — Închiderea C-09

Nivel C. `verified_confidence` null e singurul blocaj OSaaS declarat. Trebuie rezolvat înainte de desfășurare, nu după, pentru că e exact tipul de lucru care se amână la infinit odată ce sistemul e live.

P5-2 — Integrarea celor șaisprezece ramuri locale

Nivel C, cere merge. Șaisprezece ramuri locale pe laptop conțin muncă terminată neintegrată. `main` e la `44f3798`, laptopul la `bcd9f9f`. Se decide ramură cu ramură, pe baza tabelului de la P1-5: se integrează, sau se abandonează explicit. Ce nu e nici integrat, nici abandonat, devine datorie invizibilă.

Merge-ul rămâne uman. Cer acordul de fiecare dată, pentru fiecare ramură.

P5-3 — Desfășurarea

Nivel C. Pe Hetzner mai întâi, care e gazda de facto, nu pe DigitalOcean, care e căzută. Cu revizie declarată, cu posibilitate de revenire la Gen 1.5 fără pierdere de stare, și fără oprirea planurilor Python până când echivalentul Gen 2 e dovedit funcțional în paralel. Nu o migrare, o rulare în paralel urmată de o comutare.

P5-4 — Reînvierea sau retragerea gazdei DigitalOcean

Nivel B și decizie. `do-frankfurt: down, planesHealthy 0/8`, ramură veche cu aproximativ 25 de commituri în urmă, Postgres public. Ori se reface și se aduce la `main`, ori se retrage și se închide Postgres. Starea actuală e cea mai proastă: plătită, nefolositoare, expusă.

Poarta cinci se declară închisă când Gen 2 servește trafic real pe cel puțin o gazdă, cu revizie cunoscută, iar Gen 1.5 e oprit deliberat sau păstrat deliberat, nu din inerție.

8. Poarta șase — CIDA și baza de cunoaștere unică

Raportul de conformitate CIDA a dat verdictul **de returnat pentru revizuire**, cu 16 neconformități — cinci critice, șapte grave, patru medii — față de 14 constatări favorabile. Nu se promovează public înainte de închiderea Porții zero, pentru că cinci dintre neconformitățile critice sunt exact expunerile de la Poarta zero.

P6-1 — Neconformitățile critice K-01...K-05

K-05 este regula de captare generală din Caddy, care publică aproximativ douăsprezece planuri. Se închide la P0-1 și P0-2. Restul se revaluează după.

P6-2 — Baza de cunoaștere unică

Doctrina din 6 august cere o singură bază de cunoaștere partajată între suprafața principală și canalul mobil. Astăzi cunoașterea e împrăștiată: Qdrant cu colecția `ronor`, schema `cida` din Postgres, `r-memory`, plus 89 de pagini de wiki personal și 25 de sesiuni indexate care trăiesc exclusiv în afara sistemului.

P6-3 — Toate cele treisprezece proiecte de lucru au zero fișiere atașate și cunoaștere dezactivată

Nivel B, execut la cerere. Aceasta explică de ce reconstruiți contextul manual la fiecare sesiune. Nu e o omisiune de lectură, e o stare de configurare — și e reparabilă. Cele cinci proiecte nominalizate au între una și două sesiuni fiecare, iar tot contextul lor trăiește în transcrieri, nu în cunoaștere structurată. Activarea cunoașterii pe proiecte și încărcarea documentelor canonice ar elimina munca de reorientare de la începutul fiecărei sesiuni.

P6-4 — Atribuirea AI

C-11: absentă. Într-un sistem care produce rapoarte de diligență, lipsa atribuirii pentru contribuția automatizată e o problemă de conformitate, nu de stil.

P6-5 — 43 din 47 de fișiere .md sunt învechite

C-14. Documentația descrie un sistem care nu mai există. Costul nu e cosmetic: e motivul pentru care I-07 a apărut — documentația și codul spun lucruri diferite despre unde stă cheia de arhitect, și nu se poate ști care are dreptate fără să citești ambele.

9. Ce înseamnă „RONOR finalizat"

Un plan de finalizare fără criterii verificabile e o listă de intenții. Definiția pe care o propun, per suprafață:

Suprafață	Criteriu de finalizare, verificabil prin apel
Rădăcina publică	servește ce a fost decis la P2-1, nu un instrument terț; nicio cale operațională nu răspunde 200 neautentificat
Suprafața principală	una singură, versionată, cu supervisor, autentificată, cu revizie afișată
Interfața de arhitect	servită din repository la <code>/control</code> , cheia nu trece prin stocarea browserului, rutele din documentație corespund rutelor din cod
RONOR Bot	răspunde la opt comenzi, streaming nativ, co-semnare între doi identificatori distincți, supervisor, declarat în compose
„RONOR App”	declarat în <code>docs/interface-plan.md</code> — implementat sau eliminat din vocabular; niciun termen fără referent
Runtime	Gen 2 servește trafic real, <code>verified_confidence</code> populat, Gen 1.5 oprit sau păstrat deliberat
Gazde	fiecare gazdă are rol declarat, revizie cunoscută, și niciun port public nedorit; Contabo controlată sau retrasă
Cunoaștere	o singură bază canonică; contextul nu se reconstruiește manual la fiecare sesiune
Documentație	<code>docs/</code> descrie sistemul care rulează; niciun document nu contrazice codul
CIDA	neconformitățile critice și grave închise; verdictul se poate reemite

10. Ce pot executa și ce cere acordul dumneavoastră

Execut imediat, fără să întreb — nivel A: orice sondare, orice citire de configurație sau de cod, tabelul de divergență de la P1-5, diagnosticul Contabo, reverificarea C-01 până la C-03 pe celelalte gazde.

Execut integral la un singur acord, per pachet — nivel B: P0-1 închiderea celor treisprezece căi, P0-4 codurile de recuperare, P1-1 până la P1-3 aducerea sub versionare, P2-2, P3-3 până la P3-5, P3-7, P6-3.

Cer acordul explicit de fiecare dată — nivel C: P0-3 rotația cheilor, P2-3, P2-5, P3-2, P3-6, P5-1 până la P5-3. Merge, push pe `main`, release și deploy rămân umane, fără excepție.

Nu pot executa, sunt ale dumneavoastră: deciziile de la P2-1, P2-4, P3-6 despre al doilea semnatar, P5-4, și orice acces la consolele de furnizor — Hetzner, Contabo, DigitalOcean, GitHub — care nu se accesează de pe IP rusec.

Prima acțiune propusă

P0-1. Un pachet, o singură intervenție, treisprezece expuneri închise, backup înainte, verificare empirică după, revenire într-un pas. Nu atinge niciun serviciu care rulează, nu citește nicio credențială, nu modifică nicio linie de cod.

11. Riscurile planului însuși

Trei, declarate ca să nu apară ca surprize.

Unu. P0-1 poate rupe un apel intern nedescoperit. Dacă un serviciu apelează una dintre cele treisprezece căi prin adresa publică în loc de `127.0.0.1`, se va opri. Nu pot exclude asta din citirea configurației. De aceea pasul de verificare include și jurnalele Caddy, și de aceea revenirea e o singură comandă. Riscul e mic și detectabil imediat; riscul de a lăsa poarta de modele deschisă e continuu și invizibil.

Doi. Poarta doi poate rămâne blocată pe o decizie. P2-1 nu are răspuns tehnic. Dacă rămâne nedecisă, Poarta trei se poate face oricum, dar Poarta cinci nu, iar sistemul rămâne cu două suprafețe și niciun plan. Aceasta e singura poartă unde blocajul nu depinde de mine.

Trei. Ordinea Porții trei nu e negociabilă și e tentant să fie. Cel mai atrăgător pas din tot planul e să porniți botul acum, ca să vedeți că merge. Dacă se face înaintea P3-1 și P3-2, se produce al doilea conflict `getUpdates`, iar diagnosticul devine mai

scump decât așteptarea. Cincisprezece zile de moarte neobservată s-au întâmplat pentru că nimeni nu se uita; o pornire prematură reproduce exact condiția.

Toate valorile din acest document au fost măsurate direct: configurația Caddy citită integral prin [ssh](#) pe tailnet, cele 17 căi sondate prin apeluri HTTP publice, codul citit în clona locală la revizia [44f3798](#). Nicio valoare de secret și niciun hash de parolă nu apar aici. Nicio operațiune de scriere nu a fost efectuată pe depozit sau pe sistemele de producție în cursul elaborării acestui plan.